

ՀԱՅԱՍՏԱՆԻ ՀԱՆՐԱՊԵՏՈՒԹՅԱՆ ԿՐԹՈՒԹՅԱՆ, ԳԻՏՈՒԹՅԱՆ, ՄՇԱԿՈՒՅԹԻ ԵՎ
ՍՊՈՐՏԻ ՆԱԽԱՐԱՐՈՒԹՅՈՒՆ
ՀԱՅԱՍՏԱՆԻ ԱԶԳԱՅԻՆ ՊՈԼԻՏԵԽՆԻԿԱԿԱՆ ՀԱՄԱԼՍԱՐԱՆ (ՀԻՄՆԱԴՐԱՄ)
ՎԱՆԱԶՈՐԻ ՄԱՍՆԱՃՅՈՒՂ

Ամբիոն՝
Մաթեմատիկա և ծրագրային ճարտարագիտություն
Մասնագիտություն՝ Ծրագրային ճարտարագիտություն
Առարկա՝
Տվյալների հենքերի նախագծում

ՀԱՇՎԵԲԱՑԱՏՐԱԳԻՐ

կուրսային աշխատանքի/նախագծի/հետազոտական աշխատանքի
Թեմա՝ Նախագծերի և առաջադրանքների կառավարման համակարգի
տվյալների հենքի նախագծում (Project & Task Management System):

Ակադեմիական խումբ՝ ՎԾՃ 308բ

Ուսանող՝

/ Առաքելյան Արամ /
(ստորագրություն)

(ազգանուն, անուն, հայրանուն)

Ղեկավար՝

/ Ղազարյան Ս. Ս. /
(ստորագրություն)

(ազգանուն, անուն, հայրանուն)

Ամբիոնի վարիչի պ/կ՝

/ Ղազարյան Ս. Ս. /
(ստորագրություն)

(ազգանուն, անուն, հայրանուն)

ՀԱՅԱՍՏԱՆԻ ԱԶԳԱՅԻՆ ՊՈԼԻՏԵԽՆԻԿԱԿԱՆ ՀԱՄԱԼՍԱՐԱՆ
(ՀԻՄՆԱԴՐԱՄ)

Մաթեմատիկա և ծրագրային ճարտարագիտություն
ամբիոն
(ամբիոնի անվանումը)

Ծրագրային ճարտարագիտություն
մասնագիտություն
(մասնագիտության անվանումը)

ՎԾՃ 308բ
ակադեմիական խումբ

ԿՈՒՐՍԱՅԻՆ ՆԱԽԱԳԾԻ/ԱՇԽԱՏԱՆՔԻ/ՀԵՏԱԶՈՏԱԿԱՆ ԱՇԽԱՏԱՆՔԻ
ԱՌԱՋԱԴՐԱՆՔ

Տվյալների հենքերի նախագծում
(առարկայի անվանումը)

Առաքելյան Արամ
(ուսանողի ազգանուն, անուն, հայրանուն)

1. Աշխատանքի թեման Նախագծերի և առաջադրանքների կառավարման համակարգի տվյալների հենքի նախագծում (Project & Task Management System):
2. Աշխատանքի նախնական տվյալները Տարբերակ № 14
3. Հաշվեքաջատրագրի բովանդակությունը`
 - 3.1. Բովանդակություն
 - 3.2. Ներածություն
 - 3.3. Հիմնական մաս (գլուխ, բաժին, ենթաբաժին)
 - 3.4. Եզրակացություն
 - 3.5. Օգտագործված գրականության ցանկ
4. Գրաֆիկական մասի ծավալ
5. Կատարման ժամանակացույցը
6. Աշխատանքի ղեկավար
/ Ղազարյան Ս. Ս. /
(ստորագրություն) (ազգանուն, անուն, հայրանուն)
7. Ամբիոնի վարիչի պ/կ
/ Ղազարյան Ս. Ս. /
(ստորագրություն) (ազգանուն, անուն, հայրանուն)
8. Ուսանող
(ամսաթիվ, ուսանողի ստորագրություն)

ԲՈՎԱՆԴԱԿՈՒԹՅՈՒՆ

Ներածություն	4
Տվյալների հենքերի մասին ընդհանուր տեղեկություն	5
Տվյալների հենքի դերը նախագծերի կառավարման համակարգերում	5
Տվյալների հենքի կառավարման համակարգ	6
Հարաբերական տվյալների հենք	6
Ի՞նչ է PostgreSQL-ը	7
Ինչպե՞ս է օգտագործվում PostgreSQL-ը	7
PostgreSQL-ի առավելությունները	8
Ինչո՞վ են նման PostgreSQL-ը և MySQL-ը	8
PostgreSQL vs MySQL. Որո՞նք են տարբերությունները	9
Window Function	9
Կանոնավոր արտահայտությունների կիրառումը PostgreSQL-ում	10
Տվյալների բազայի միգրացիա (Database Migration)	10
Հիմնական նկատառումներ	10
Տվյալների հենքի նախագծում	11
View, Stored Procedure և Trigger	11
Տրիգերներ	12
Տրիգերի բաղադրիչները	12
Տրիգերների տիպերը	13
Տրիգերների ստեղծում	13
Տրիգերների ծրագրավորում	14
Տվյալների անվտանգություն	15
Authentication and Authorization Using JWT	16
Տվյալների նորմալացում	16
Օգտագործված գործիքներ և տեխնոլոգիաներ	17
PostgreSQL 16 — տվյալների բազայի սերվեր	17
FastAPI և SQLAlchemy 2 (async) — backend	17
React 18 + TypeScript + Vite — frontend	17
Git և GitHub — տարբերակների կառավարում	17
Docker և Docker Compose — միջավայրի կազմակերպում	17
Alembic migration և seed հրամանների իրականացում	17
PostgreSQL սերվերի գործարկում Docker-ով	18
PostgreSQL psql միջավայրի միացում	18
PostgreSQL տվյալների բազայի աղյուսակների դիտարկում	18
Կոդի խմբագրիչ (Visual Studio Code)	19
Render.com — ամպային տեղակայում	19
Ծրագրային կոդեր	19
Backend	19
Frontend	25
Եզրակացություն	28
Գրականության ցանկ	28

Ներածություն

Ժամանակակից տեղեկատվական համակարգերում նախագծերի և առաջադրանքների կառավարման հարթակները դարձել են ծրագրային նախագծերի կազմակերպման, թիմային աշխատանքի համակարգման և կատարման արդյունավետության բարձրացման անբաժանելի մաս: Ընկերություններում ամեն օր ստեղծվում և մշակվում է մեծ ծավալի տեղեկատվություն, որը ներառում է նախագծերի նկարագրությունները, առաջադրանքները, պատասխանատուներին, կատարման ժամկետները, կարգավիճակները, մեկնաբանությունները և գործողությունների պատմությունը: Այս տեղեկատվության արդյունավետ կառավարումը պահանջում է հուսալի և անվտանգ տվյալների հենքի առկայություն, որը կապահովի տվյալների ամբողջականությունը, արագ հասանելիությունը և ճշգրտությունը:

Տվյալների հենքերի կառավարման համար լայնորեն կիրառվում է SQL (Structured Query Language) լեզուն, որը նախատեսված է տվյալների ստեղծման, պահպանման, փոփոխման և մշակման համար: SQL-ը հանդիսանում է հարաբերական տվյալների բազաների հետ աշխատանքի հիմնական ստանդարտ գործիքներից մեկը և հնարավորություն է տալիս իրականացնել տվյալների արդյունավետ որոնում, վերլուծություն և կառավարում: Այս աշխատանքում տվյալների բազայի կառավարման համար օգտագործվել է PostgreSQL համակարգը, որը SQL լեզվի վրա հիմնված, բաց կոդով և բարձր հուսալիությամբ օժտված տվյալների բազայի կառավարման միջավայր է: PostgreSQL-ը առանձնանում է տվյալների ամբողջականության պահպանման բարձր մակարդակով, մեծածավալ տվյալների արդյունավետ մշակման հնարավորություններով և լայն կիրառությամբ տարբեր տեղեկատվական համակարգերում, այդ թվում՝ ծրագրային նախագծերի կառավարման ոլորտում:

Սույն կուրսային աշխատանքի նպատակն է նախագծել նախագծերի և առաջադրանքների կառավարման համակարգի (Project & Task Management System, PTMS) տվյալների հենքը՝ ապահովելով օգտատերերի, նախագծերի, առաջադրանքների, ենթաառաջադրանքների, թեգերի, մեկնաբանությունների, ինչպես նաև գործողությունների պատմության արդյունավետ պահպանումը և կառավարումը: Աշխատանքի շրջանակներում իրականացվել է տվյալների հենքի կառուցվածքի նախագծում, սահմանվել են աղյուսակները, հիմնական և արտաքին բանալիները, ինդեքսները, ստուգման սահմանափակումները ապահովելով տվյալների ամբողջականության պահպանման պահանջները:

Արդյունքում մշակվել է նախագծերի և առաջադրանքների կառավարման համակարգի տվյալների հենքի ամբողջական նախագիծ, որը կարող է հիմք ծառայել ծրագրային նախագծերի կառավարման տեղեկատվական համակարգի ներդրման համար և նպաստել կազմակերպության աշխատանքի ավտոմատացմանը, արդյունավետության բարձրացմանը և տվյալների կառավարման որակի բարելավմանը:

Տվյալների հենքերի մասին ընդհանուր տեղեկություն

Մարդն իր գործունեության ընթացքում սովորել է ոչ միայն ինֆորմացիա հավաքել, այլև այն համակարգել: Որպեսզի կուտակված ինֆորմացիան մատչելի լինի, այն նախ հատուկ ձևով տեսակավորվել է, ապա համակարգվել տեղեկագրերում: Նման տեղեկագրեր են, օրինակ, կյանքի ամենաբազմազան ոլորտներին վերաբերող հանրագիտարանները: Որպեսզի նման գրականության միջոցով տեղեկություններ ստանաք, ասենք, որևէ եզակի առարկայի վերաբերյալ, հավանաբար պետք է մասնագիտացված գրադարան գտնել, և հազարավոր գրքերից փնտրել պետք եղած ինֆորմացիան: Իսկ եթե նման գրադարան քաղաքում չկա, կամ էլ համապատասխան ինֆորմացիան անհրաժեշտ է շտապ ձեռք բերել, ապա, բնականաբար, անելանելի իրավիճակ կստեղծվի, եթե չօգտվենք համակարգչային տեխնոլոգիաներից:

Գոյություն ունեն համակարգչային տարբեր ծրագրային միջոցներ, որոնք թույլատրում են փաստաթղթեր ու գրքեր ստեղծել, հաշվարկներ կատարել, գրաֆիկական բարդ օբյեկտներ ստեղծել և այլն: Մեծ քանակությամբ ինֆորմացիա պահպանելու և մշակելու նպատակով ևս հատուկ ծրագրային միջոցներ են ստեղծվել, որոնք կոչվում են հենքային (բազային) տվյալների ղեկավարման համակարգեր: Տվյալների հենք (որոշակի նպատակով կառուցվածքայնացված տվյալների համախումբն անվանում են տվյալների հենք) հասկացության ներքո պետք է ընդունել յուրահատուկ եղանակով կազմակերպված այնպիսի ինֆորմացիայի ամբողջությունը, որը պահպանվում է որևէ նյութական կրիչի վրա: Կարևոր է նաև հասկանալ, որ հենքերի մեջ կուտակվածն այնպիսի կապակցված ինֆորմացիա է, որը թույլատրում է հստակ պատկերացում կազմել որևէ օբյեկտի, երևույթի կամ գործընթացի վերաբերյալ: Օրինակ՝ նախագծերի կառավարման տվյալների հենքի մեջ զանազան ինֆորմացիա պետք է լինի նախագծերի, առաջադրանքների, կատարողների և կարգավիճակների վերաբերյալ, բայց չի կարող տեղեկություն լինել, օրինակ, որևէ խանութի հաշվապահության մասին:

Տվյալների միևնույն հենքի մեջ եղած տեղեկությունները պետք է թե՛ իմաստով, և թե՛ տրամաբանությամբ իրար կապակցված լինեն: Դրա համար տվյալների հենքին ուղղվող ինֆորմացիան խմբավորվում է ըստ որոշակի պարամետրերի: Նման գործընթացն անվանում են տվյալների կառուցվածքայնացում: Տվյալների հենքերի հետ աշխատելու համար ստեղծված ծրագրային միջոցներն անվանում են տվյալների հենքերի ղեկավարման համակարգին (ՏՀՂՀ) ուղղված ծրագրեր:

Ժամանակակից տվյալների հենքերի Oracle, SQL Server, MySQL, PostgreSQL ղեկավարման համակարգերում, պարզ տվյալներից բացի, պահպանվում են նաև ծրագրային միջոցներ, այլ կերպ ասած՝ մեթոդներ, ընթացակարգեր, ֆունկցիաներ և տրիգերներ:

Տվյալների հենքի դերը նախագծերի կառավարման համակարգերում

Ժամանակակից տեղեկատվական համակարգերում տվյալների հենքերը կարևոր դեր են խաղում տեղեկատվության պահպանման, մշակման և կառավարման գործընթացում: Ծրագրային նախագծեր իրականացնող կազմակերպություններում օրական ստեղծվում և մշակվում է մեծ քանակությամբ տվյալներ, որոնք վերաբերում

են նախագծերին, առաջադրանքներին, օգտատերերին, կարգավիճակների փոփոխությանը, մեկնաբանություններին և գործողություններին պատմությանը:

Այս տվյալների արդյունավետ կառավարման համար անհրաժեշտ է օգտագործել տվյալների հենքի կառավարման համակարգեր, որոնք ապահովում են տվյալների անվտանգ պահպանում, արագ որոնում և հարմար մշակման հնարավորություն: Տվյալների հենքերի կիրառումը PTMS համակարգում թույլ է տալիս ավտոմատացնել նախագծերի կառավարման գործընթացները, նվազեցնել սխալների հավանականությունը և բարձրացնել թիմի աշխատանքի արդյունավետությունը:

PTMS համակարգերը հանդիսանում են տեղեկատվական համակարգեր, որոնք նախատեսված են ծրագրավորողների, նախագծերի մենեջերների և թիմի անդամների աշխատանքը կազմակերպելու և առաջադրանքների կառավարման համար: Այդպիսի համակարգերը հնարավորություն են տալիս արագ հասանելիություն ունենալ նախագծի տվյալներին, ստեղծել առաջադրանքներ, սահմանել կատարման ժամկետներ, նշանակել կատարողներ և հետևել ընթացքին:

Տվյալների հենքի կառավարման համակարգ

Տվյալների հենքի կառավարման համակարգը (Database Management System – DBMS) ծրագրային համակարգ է, որը նախատեսված է տվյալների հենքերի ստեղծման, կառավարման և օգտագործման համար: Այն ապահովում է տվյալների պահպանումը, անվտանգությունը և արդյունավետ օգտագործումը տարբեր կիրառական ծրագրերի կողմից:

Տվյալների հենքի կառավարման համակարգերի հիմնական գործառույթներն են՝

- տվյալների պահպանում
- տվյալների փոփոխում
- տվյալների որոնում
- տվյալների անվտանգություն
- միաժամանակյա օգտագործման կառավարում

Տվյալների հենքի կառավարման համակարգերը լայնորեն կիրառվում են տարբեր ոլորտներում, ինչպիսիք են բանկային համակարգերը, կրթական հաստատությունները, ծրագրային նախագծերի կառավարման հարթակները և պետական կառույցները:

Այս կուրսային աշխատանքում տվյալների հենքը նախագծվում է օգտագործելով PostgreSQL միջավայրը, որը հանդիսանում է աշխարհում լայնորեն կիրառվող օբյեկտ-հարաբերական տվյալների հենքի կառավարման համակարգերից մեկը:

Հարաբերական տվյալների հենք

Հարաբերական տվյալների հենքը տվյալների կազմակերպման մոդել է, որտեղ տվյալները պահվում են աղյուսակների տեսքով: Յուրաքանչյուր աղյուսակ բաղկացած է տողերից և սյունակներից, որտեղ սյունակները ներկայացնում են տվյալների հատկանիշները, իսկ տողերը կոնկրետ գրառումները:

Հարաբերական տվյալների հենքերում աղյուսակները կարող են կապվել միմյանց հետ հատուկ բանալիների միջոցով: Այդ կապերը թույլ են տալիս կազմակերպել տվյալները այնպես, որպեսզի դրանք լինեն կառուցվածքային, ամբողջական և հեշտ օգտագործելի:

Հարաբերական տվյալների հենքերի հիմնական տարրերն են՝

Աղյուսակ (Table) – տվյալների հիմնական կառուցվածքային միավորն է, որտեղ պահվում են տվյալները:

Տող (Row) – ներկայացնում է տվյալների մեկ գրառում:

Սյունակ (Column) – ներկայացնում է տվյալների կոնկրետ հատկանիշը:

Առաջնային բանալի (Primary Key) – եզակի դաշտ է, որը նույնականացնում է աղյուսակի յուրաքանչյուր գրառում:

Արտաքին բանալի (Foreign Key) – դաշտ է, որը ապահովում է կապ երկու աղյուսակների միջև:

Ի՞նչ է PostgreSQL-ը

PostgreSQL-ը, որը հայտնի է նաև որպես Postgres, բաց կոդով հարաբերական տվյալների բազայի կառավարման համակարգ է, որը առանձնանում է իր բարձր հուսալիությամբ, ճկունությամբ և բաց տեխնիկական ստանդարտների աջակցությամբ: Այն համարվում է SQL ստանդարտին համապատասխանող և առավել կայուն ու հասուն տվյալների բազաներից մեկը, որը լայնորեն կիրառվում է ժամանակակից տեղեկատվական համակարգերում: Այն հնարավորություն է տալիս արդյունավետ կերպով աշխատել ինչպես պարզ, այնպես էլ բարդ կառուցվածք ունեցող հարցումների հետ ապահովելով տվյալների արագ և անվտանգ մշակում:

Բացի հիմնական հարաբերական մոդելից, PostgreSQL-ը աջակցում է նաև ոչ հարաբերական տվյալների տեսակների, օրինակ՝ JSON և JSONB, ինչը այն դարձնում է առավել ճկուն և հարմար տարբեր տեսակի կիրառությունների համար:

PostgreSQL-ի առանձնահատկությունները ներառում են հետևյալը.

- Ժամանակի կետում վերականգնում (PITR)՝ տվյալների բազաները ժամանակի որոշակի պահի վերականգնելու համար:
- Գրեք նախնական գրանցամատյան (WAL), որը գրանցում է տվյալների բազայում կատարված բոլոր փոփոխությունները pgBackRest-ի նման գործիքների միջոցով:
- Պահպանված պրոցեդուրաներ՝ հատուկ ենթածրագրեր ստեղծելու և պահպանելու համար:
- MVCC՝ բազմատարբերակային գույքահեռականության կառավարման մեխանիզմ, որը հնարավորություն է տալիս միաժամանակ կատարել գրելու և կարդալու գործառնություններ:

Ինչպե՞ս է օգտագործվում PostgreSQL-ը

PostgreSQL-ը «միասնական լուծում է» բազմաթիվ ձեռնարկությունների համար, որոնք

փնտրում են իրենց տվյալների բազայի կառավարման համակարգերը (DBMS) բարելավելու ծախսարդյունավետ և արդյունավետ եղանակներ: Այն ընդարձակվող է և բավականաչափ բազմակողմանի՝ արագորեն աջակցելու տարբեր մասնագիտացված օգտագործման դեպքերի՝ հզոր ընդլայնման էկոհամակարգով, որը ներառում է ժամանակային շարքերի տվյալների տեսակների և աշխարհագրական վերլուծությունների նման ջանքեր: Կառուցված որպես բաց կոդով տվյալների բազայի լուծում, PostgreSQL-ը լիովին ազատ է լիցենզավորման սահմանափակումներից, մատակարարի կողմից կապվածության ներուժից կամ գերտեղակայման ռիսկից: PostgreSQL-ը կառավարվում է օբյեկտ-ռելացիոն տվյալների բազայի կառավարման համակարգի (ORDBMS) միջոցով:

PostgreSQL-ի առավելությունները

Սրանք PostgreSQL-ի հիմնական առավելություններից մի քանիսն են՝

- Կատարողականություն և մասշտաբայնություն, ներառյալ աշխարհատարածական աջակցությունը և անսահմանափակ համաժամանակությունը, և տվյալների խորը, ընդարձակ վերլուծությունը տվյալների բազմաթիվ տեսակների միջև:
- Միաժամանակյա աջակցություն բազմատեսակ միաժամանակյա հսկողության (MVCC) օգտագործման միջոցով, որը հնարավորություն է տալիս միաժամանակ կատարել գրելու գործառնություններ և կարդալ:
- Խորը լեզվի աջակցություն՝ շնորհիվ իր համատեղելիության և ծրագրավորման բազմաթիվ լեզուների՝ ներառյալ Python, Java, JavaScript, C/C++ և Ruby:
- Գործարար շարունակականություն՝ ծառայությունների բարձր հասանելիությամբ սերվերներում ասինխրոն կամ համաժամանակյա վերաբաշխման մեթոդների միջոցով:
- Ավելի մեծ ճկունություն և ծախսարդյունավետ նորարարություն՝ բաց կոդով տվյալների բազայի կառավարման տեխնոլոգիայի միջոցով:

Ինչո՞վ են նման PostgreSQL-ը և MySQL-ը

Թե՛ PostgreSQL-ը, թե՛ MySQL-ը հիմնված են SQL-ի վրա, որը կառավարման համակարգերի հետ փոխազդեցության ստանդարտ լեզուն է: SQL-ը հնարավորություն է տալիս միացնել աղյուսակները՝ օգտագործելով մի քանի տող սկզբնական կոդ՝ պարզ կառուցվածքով:

SQL-ի միջոցով վերլուծաբանները կարիք չունեն իմանալու, թե որտեղ է գտնվում առաջադրանքների աղյուսակը սկավառակի վրա, ինչպես կատարել որոնումը՝ որոշակի առաջադրանք գտնելու համար կամ ինչպես կապել առաջադրանքների և օգտատերերի աղյուսակները: Տվյալների բազան կազմում է հարցումը և պարզում է ճիշտ տվյալները:

Ե՛վ MySQL-ը, և՛ PostgreSQL-ը աջակցում են JavaScript Object Notation (JSON)-ին՝ տվյալները պահելու և տեղափոխելու համար, չնայած PostgreSQL-ը նաև աջակցում է JSONB-ին՝ JSON-ի երկուական տարբերակին, որը վերացնում է բանալիների

կրկնօրինակումը և ավելորդ սպիտակ տարածությունները:

PostgreSQL vs MySQL. Որո՞նք են տարբերությունները

PostgreSQL-ի և MySQL-ի միջև կան բազմաթիվ տարբերություններ: Հատկանիշների, ֆունկցիոնալության և առավելությունների որոշ տարբերությունները հետևյալն են՝

- Տվյալների բազայի տեսակը — MySQL: Հարաբերական, PostgreSQL: Օբյեկտ-ռելացիոն
- Ծրագրավորման լեզու — MySQL: C/C++, PostgreSQL: C
- CASCADE-ի աջակցություն — MySQL: Ոչ, PostgreSQL: Այո
- Օգտագործողի ինտերֆեյս — MySQL: MySQL Workbench, PostgreSQL: pgAdmin
- Աջակցվող ընթացակարգի բարդությունը — MySQL: SQL սինտաքսներ և պահված պրոցեդուրաներ, PostgreSQL: Ընդլայնված և պահված ընթացակարգեր
- Աջակցվող ինդեքսի տեսակը — MySQL: B-ծառ, PostgreSQL: Շատերը, այդ թվում՝ GIN, GiST, BRIN և Hash
- Հաճախորդ-սերվեր կոդավորում — MySQL: TLS, PostgreSQL: SSL
- XML տվյալների տեսակ — MySQL: Ոչ, PostgreSQL: Այո
- Materialized View և աղյուսակի ժառանգում — MySQL: Ոչ, PostgreSQL: Այո
- Առաջադեմ տվյալների տեսակներ — MySQL: Ոչ, PostgreSQL: Այո (hstore և օգտատիրոջ կողմից սահմանված տիպեր)
- MVCC — MySQL: Ոչ, PostgreSQL: Այո

Window Function

Window Functions-ը SQL լեզվի վերլուծական ֆունկցիաների խումբ են, որոնք հնարավորություն են տալիս կատարել հաշվարկներ տվյալների որոշակի «պատուհանի» (window) սահմաններում՝ առանց արդյունքային տողերի քանակը նվազեցնելու կամ տվյալների համախմբում իրականացնելու:

Ի տարբերություն ագրեգատ ֆունկցիաների, window functions-ը հաշվարկները կատարում են յուրաքանչյուր տողի մակարդակում՝ պահպանելով տվյալների սկզբնական կառուցվածքը և միաժամանակ ապահովելով լրացուցիչ վերլուծական արժեքներ:

Դրանք կիրառվում են OVER() կառուցվածքի միջոցով, որը կարող է ներառել PARTITION BY (խմբավորում) և ORDER BY (դասավորում) բաղադրիչները, ինչպես նաև անհրաժեշտության դեպքում՝ frame clause:

Window functions-ը լայնորեն կիրառվում են դասակարգման, կուտակային հաշվարկների և ժամանակային վերլուծությունների իրականացման համար՝ ապահովելով տվյալների ավելի խորացված և ճշգրիտ մշակման հնարավորություն: PTMS համակարգում window functions-ը կիրառվել են առաջադրանքների առաջընթացի ցուցադրման և թիմի անդամների արտադրողականության հաշվարկման համար:

Կանոնավոր արտահայտությունների կիրառումը PostgreSQL-ում

Կանոնավոր արտահայտությունները (Regular Expressions, regex) տեքստային տվյալների որոնման և համապատասխանեցման մեխանիզմ են, որոնք հիմնված են նախապես սահմանված նմուշների վրա: Դրանք հնարավորություն են տալիս արդյունավետ կերպով իրականացնել բարդ տեքստային որոնումներ, տվյալների վավերացում և մշակման գործողություններ:

PostgreSQL-ում կանոնավոր արտահայտությունները լայնորեն կիրառվում են տեքստային դաշտերում համապատասխան արժեքների որոնման և կառուցվածքային ստուգման համար՝ ապահովելով բարձր ճշգրտություն և ճկունություն: Կանոնավոր արտահայտությունները հատկապես օգտակար են այն դեպքերում, երբ անհրաժեշտ է գտնել որոշակի ձևաչափ ունեցող տվյալներ, իրականացնել մուտքային տվյալների վավերացում կամ ստուգել տվյալների համապատասխանությունը սահմանված կանոններին:

Շարահյուսություն՝

```
SELECT column_name
FROM table_name
WHERE column_name ~ 'pattern';
```

~ օպերատորը օգտագործվում է տվյալ սյունակի արժեքը տվյալ կանոնավոր արտահայտության հետ համեմատելու համար և վերադարձնում է այն տողերը, որոնք համապատասխանում են նշված նմուշին: PTMS համակարգում regex-ը կիրառվել է էլեկտրոնային փոստի և օգտատիրոջ անվան վավերացման համար:

Տվյալների բազայի միգրացիա (Database Migration)

Տվյալների բազայի միգրացիան գործընթաց է, որի ընթացքում տվյալները տեղափոխվում են սկզբնաղբյուր տվյալների բազայից դեպի նպատակային տվյալների բազա: Այն կարող է ներառել տարբեր DBMS-ների միջև տեղափոխում կամ նույն համակարգի նոր տարբերակի անցում: Միգրացիան կիրառվում է համակարգերի արդիականացման, արտադրողականության բարձրացման և ամպային լուծումների անցման նպատակներով: PTMS համակարգում տվյալների բազայի սխեմայի կառավարման համար օգտագործվում է Alembic միգրացիոն գործիքը՝ SQLAlchemy ORM-ի հետ միասին:

Հիմնական նկատառումներ

Սխեմայի միգրացիա (Schema Migration)

Ապահովում է տվյալների բազայի կառուցվածքի համապատասխանեցումը նպատակային համակարգին ներառելով աղյուսակների և դաշտերի փոփոխություն:

Տվյալների ամբողջականություն (Data Consistency)

Պահպանում է տվյալների ճշգրտությունն ու համադրելիությունը միգրացիայի ընթացքում՝ կանխելով կորուստներն ու հակասությունները:

Միգրացիայի ռազմավարություններ՝

- Big Bang – ամբողջական տեղափոխում միանգամից
- Trickle – աստիճանական տեղափոխում
- Zero-downtime – առանց աշխատանքի ընդհատման միգրացիա

Թեստավորում և վավերացում

Ապահովում է տվյալների, կապերի և ինդեքսների ճիշտ փոխանցումը նպատակային տվյալների բազա:

Տվյալների հենքի նախագծում

Տվյալների հենքի նախագծումը կարևոր փուլ է տեղեկատվական համակարգի ստեղծման ընթացքում: Նախագծման նպատակն է ստեղծել տվյալների այնպիսի կառուցվածք, որը կլինի արդյունավետ, ճկուն և հեշտ ընդլայնելի:

Տվյալների հենքի նախագծման հիմնական փուլերն են

1. Համակարգի պահանջների վերլուծություն
2. Հիմնական entity-ների որոշում
3. Աղյուսակների կառուցվածքի նախագծում
4. Կապերի սահմանում աղյուսակների միջև
5. Տվյալների ամբողջականության ապահովում

PTMS-ը (Project & Task Management System) տեղեկատվական համակարգ է, որը նախատեսված է ծրագրավորող թիմերի, նախագծերի մենեջերների և ընկերության աշխատանքը կազմակերպելու և հեշտացնելու համար: Այս համակարգի միջոցով օգտատերերը կարող են արագ ստեղծել նախագծեր, ձևավորել առաջադրանքների ցանկը, ընտրել կատարողներ, սահմանել առաջնահերթություններ և հետևել նախագծի առաջընթացին:

PTMS համակարգի օգտագործումը թույլ է տալիս՝

- կազմակերպել նախագծերի և առաջադրանքների տվյալների պահպանումը
- ավտոմատացնել կարգավիճակների կառավարման գործընթացները
- արագ գտնել անհրաժեշտ տեղեկատվությունը
- նվազեցնել ձեռքով կատարվող փաստաթղթային աշխատանքը
- ապահովել թիմի անդամների միջև համագործակցությունը

Այս համակարգերը կարևոր դեր ունեն ժամանակակից թվային ձեռնարկությունների կառավարման զարգացման մեջ:

View, Stored Procedure և Trigger

Տվյալների հենքերում հաճախ օգտագործվում են լրացուցիչ մեխանիզմներ, որոնք օգնում են ավելի արդյունավետ աշխատել տվյալների հետ:

View-ը վիրտուալ աղյուսակ է, որը ստեղծվում է SELECT հարցման հիման վրա: Այն օգտագործվում է տվյալները ավելի հարմար ձևով ներկայացնելու համար: PTMS համակարգում, օրինակ, ստեղծվել է `v_project_progress` տեսքը, որը ցույց է տալիս

յուրաքանչյուր նախագծի առաջընթացը՝ կատարված և ընդհանուր առաջադրանքների հարաբերակցության հիման վրա:

Stored Procedure-ը SQL հրամանների հավաքածու է, որը պահվում է տվյալների հենքում և կարող է բազմիցս օգտագործվել տարբեր գործողություններ կատարելու համար:

Trigger-ը հատուկ մեխանիզմ է, որը ավտոմատ կերպով ակտիվանում է, երբ տվյալների հենքում կատարվում են որոշակի գործողություններ, օրինակ՝ տվյալների ավելացում կամ փոփոխում:

Այս մեխանիզմները մեծապես հեշտացնում են տվյալների կառավարման գործընթացը և ապահովում համակարգի արդյունավետ աշխատանքը:

Տրիգերներ

Տրիգերներն իրենցից ներկայացնում են նախապես որոշված գործողություն կամ գործողությունների հաջորդականություն, որն ավտոմատ կերպով կատարվում է, տվյալների նորացման, ավելացման կամ հեռացման գործողությունների կատարման ժամանակ: Ոչ օգտվողը, ոչ էլ ծրագրային հավելվածը չեն կարող ակտիվացնել տրիգերը, այն կատարվում է ավտոմատ կերպով, երբ օգտվողը կամ ծրագրային հավելվածը տվյալների բազայի կետ կատարում են որոշակի գործողություն: Տրիգերը պահվող պրոցեդուրայի հատուկ տեսակ է: Տրիգերներն ապահովում են տվյալների բազայի ցանկացած փոփոխության ճշտություն, նախքան այդ փոփոխությունների ընդունումը:

Յուրաքանչյուր տրիգեր կցվում է որոշակի (կոնկրետ) աղյուսակի: Նրա կողմից կատարված տվյալների բոլոր մոդիֆիկացիաները դիտարկվում են որպես մեկ տրանզակցիա: Միայն հայտնաբերման ժամանակ կամ տվյալների ամբողջականության խախտման դեպքում կատարվում է տրանզակցիայի ետ քայլ: Այդպիսով փոփոխության կատարումն արգելվում է: Չեղարկվում են նաև տրիգերի կողմից արդեն կատարված բոլոր փոփոխությունները:

Տրիգեր կարող է ստեղծել միայն տվյալների բազայի սեփականատերը: Այս սահմանափակումը թույլ է տալիս խուսափել աղյուսակի կառուցվածքի, նրա հետ այլ օբյեկտների կապի ձևերի և այլ պատահական փոփոխություններից:

Տրիգերի բաղադրիչները

1. Սահմանափակումներ, որոնց իրացման համար ստեղծվում է տրիգեր:
2. Իրադարձություն, որը պետք է բնութագրի սահմանափակման ստուգում պահանջող իրավիճակի ծագումը: Տրիգերային իրադարձությունները ավելի հաճախ կապված են տվյալների բազայի վիճակի փոփոխության հետ և կազմված են աղյուսակում տողի ավելացումից, հեռացումից և նորացումից: Իրադարձություններում կարող են հաշվի առնվել նաև լրացուցիչ պայմաններ (օրինակ, միայն բացասական արժեքներով գրառման ավելացում):
3. Նախատեսված գործողություն իրականացվում է պրոցեդուրայի կամ պրոցեդուրաների հաջորդականության կատարման հաշվին, որոնց օգնությամբ իրացվում է սահմանափակումն իրացնելու համար պահանջվող

FOR EACH { ROW | STATEMENT } կառուցվածքները որոշում են, թե տրիգերը մեկնարկելու է յուրաքանչյուր տողի համար, թե մեկ անգամ՝ ողջ հրամանի վերջում: Նրա ստեղծման ժամանակ պետք է նշված լինի հրամաններից գոնե մեկը:

Միայն գրված տրիգերները կարող են հանգեցնել լուրջ խնդիրների, ինչպիսին է, օրինակ, «մեռյալ» բլոկավորումների ի հայտ գալը: Տրիգերներն ի վիճակի են երկար ժամանակով բլոկավորել բազմաթիվ ռեսուրսներ, այդ պատճառով անհրաժեշտ է հատուկ ուշադրություն դարձնել հասանելիության հակասությունները նվազագույնի հասցնելուն:

ՏԲՂՀ-երի մեծամասնության մեջ գործում են հետևյալ սահմանափակումները՝

- Տրիգերի մարմնում չի կարելի օգտագործել տվյալների բազայի օբյեկտներ, ինչպիսիք են նոր տվյալների բազան, աղյուսակը, պահվող պրոցեդուրան, տրիգերը, ներկայացումը և այլն, ստեղծող գործողություններ:
- Տրիգերում տվյալների բազայի բոլոր տիպերի բազային օբյեկտների համար չի կարելի օգտագործել հեռացման DROP հրաման:
- Տրիգերի մարմնում չի կարելի օգտագործել բազային օբյեկտների փոփոխության ALTER TABLE, ALTER DATABASE հրամաններ:
- Չի կարելի փոփոխել տվյալների բազայի օբյեկտներին հասանելիության իրավունքները, այսինքն՝ կատարել GRANT կամ REVOKE հրամանները:
- Ի տարբերություն պահպանվող պրոցեդուրաների, տրիգերը չի կարող վերադարձնել ոչ մի արժեք: Այն ավտոմատորեն գործարկվում է սերվերի կողմից, և ինքնուրույն չի կարող կապվել ոչ մի հաճախորդի հետ:
- Տրիգերի ներսում չի թույլատրվում տվյալների բազայի կամ տրանզակցիաների մատյանի վերականգնում պատճեններից:

Տրիգերների օգտագործման առավելությունները՝

1. Տրիգերը միշտ կատարվում է համապատասխան գործողության կայացման դեպքում: Մշակողը տրիգերների օգտագործման մասին մտածում է տվյալների բազայի նախագծման ժամանակ և կարող է այլևս նրա մասին չհիշել, տվյալների հասանելիության ծրագրային հավելվածների մշակման ժամանակ:
2. Անհրաժեշտության դեպքում տրիգերները կարելի է փոփոխել կենտրոնացված ձևով, անմիջականորեն տվյալների բազայում: Օգտվողների ծրագրերը, որոնք օգտագործում են տվյալներ այդ տվյալների բազայից, ձևափոխության անհրաժեշտություն չունեն:
3. Տվյալների մշակման համակարգը, որն օգտագործում է տրիգերներ, տիրապետում է լավագույն տեղափոխելիության հաճախորդ-սերվեր ճարտարապետության մեջ՝ ի հաշիվ անհրաժեշտ փոփոխությունների նվազագույն ծավալի:

Տրիգերների ծրագրավորում

Ավելացման, փոփոխության և հեռացման հրամանների կատարման ժամանակ սերվերը PostgreSQL-ում տրամադրում է երկու հատուկ գրառումներ՝ NEW և OLD: NEW-ը պարունակում է ավելացվող կամ նորացված տողի արժեքները, իսկ OLD-ը՝

նորացված կամ հեռացված տողի նախորդ արժեքները:

Գործողության տեսակից կախված, որը մեկնարկել է տրիգերի կատարումը, NEW և OLD գրառումների պարունակությունները կարող են լինել տարբեր՝

- INSERT հրամանի դեպքում NEW գրառման մեջ պարունակվում են այն արժեքները, որոնք օգտվողը փորձում է ավելացնել աղյուսակում: OLD-ը այստեղ դատարկ է:
- DELETE հրամանի դեպքում OLD գրառման մեջ պարունակվում են այն բոլոր արժեքները, որոնք օգտվողը փորձում է հեռացնել աղյուսակից: NEW-ը այստեղ դատարկ է:
- UPDATE հրամանի կատարման ժամանակ OLD-ում պարունակվում են նախորդ արժեքները, իսկ NEW-ում՝ նոր արժեքները:

Տրիգերը կատարվում է որպես անբացահայտ որոշված տրանզակցիա, այդ պատճառով տրիգերի ներսում թույլատրվում է տրանզակցիաների դեկլարման հրամանների օգտագործում: Մասնավորապես, տրիգերի կատարման ընդհատման և բոլոր փոփոխությունների, որը փորձում էր կատարել օգտվողը, չեղարկման համար ամբողջականության սահմանափակումների խախտման բացահայտման դեպքում PL/pgSQL-ում օգտագործվում է RAISE EXCEPTION հրամանը: Տրանզակցիայի կատարման ժամանակ մտցված փոփոխությունների ամրագրման համար գործում է ավտոմատ COMMIT մեխանիզմը՝ ֆունկցիայի հաջող ավարտից հետո:

Տրիգերի հեռացման համար օգտագործվում է հետևյալ հրամանը՝

```
DROP TRIGGER <■■■■■■■■_■■■■■■■■> ON <■■■■■■■■■■_■■■■■■■■>;
```

Տվյալների անվտանգություն

Նախագծերի կառավարման տեղեկատվական համակարգերում տվյալների անվտանգությունը շատ կարևոր է, քանի որ այդ համակարգերում պահվում են ընկերության ներքին տվյալներ, օգտատերերի անձնական տեղեկություններ, ինչպես նաև գաղտնի բիզնես-տեղեկատվություն:

Տվյալների անվտանգությունը ապահովվում է տարբեր մեխանիզմների միջոցով՝

- օգտատերերի նույնականացում (authentication)
- մուտքի իրավունքների սահմանում (authorization)
- տվյալների պաշտպանություն (encryption at rest, TLS in transit)
- տվյալների պահուստավորում (backup)
- գործողությունների աուդիտ (activity log)

Տվյալների հենքի կառավարման համակարգերում հնարավոր է սահմանել տարբեր օգտատերերի իրավունքներ: Օրինակ՝ ադմինիստրատորը կարող է կառավարել ամբողջ համակարգը, նախագծի մենեջերը կարող է խմբագրել իր նախագիծը և նրա առաջադրանքները, իսկ սովորական անդամը կարող է տեսնել նախագծերը, որոնցում նա ընդգրկված է:

Այս մեխանիզմները թույլ են տալիս ապահովել տվյալների գաղտնիությունը և

կանխել չարտոնված մուտքը:

Authentication and Authorization Using JWT

JSON Web Token (JWT) ստանդարտը ժամանակակից վեբ հավելվածներում նույնականացման և թույլտվությունների կառավարման ամենատարածված մեթոդներից մեկն է: JWT-ն ներկայացնում է գաղտնագրորեն ստորագրված տոկեն, որը ստեղծվում է սերվերի կողմից օգտատիրոջ հաջող մուտքից հետո և օգտագործվում է հետագա հարցումներում օգտատիրոջ ինքնությունը հաստատելու համար:

PTMS համակարգում նույնականացման համար օգտագործվում է JWT-ի երկու տարատեսակ access token (կարճ ժամանակով) և refresh token (երկար ժամանակով, պահվում է HTTP-only cookie-ում): Access token-ը ուղարկվում է յուրաքանչյուր API հարցման հետ Authorization header-ում, իսկ refresh token-ը օգտագործվում է access token-ի թարմացման համար՝ առանց օգտատիրոջը կրկին մուտք գործելու պարտադրելու:

Թույլտվությունների (authorization) կառավարման համար կիրառվում է դերերի վրա հիմնված մուտքի վերահսկման (RBAC) մոդելը: Օգտատիրոջ ընդհանուր դերերն են՝ superadmin, admin, manager, member: Բացի այդ, յուրաքանչյուր նախագծի ներսում գործում են լրացուցիչ դերեր՝ manager, member, viewer, որոնք պահվում են project_members կապող-աղյուսակում: Այս երկշերտ մոդելը ապահովում է ճկուն և մանրամասն մուտքի վերահսկում:

Տվյալների նորմալացում

Տվյալների նորմալացումը տվյալների հենքի նախագծման կարևոր գործընթաց է, որի նպատակն է նվազեցնել տվյալների կրկնությունը և ապահովել տվյալների կառուցվածքային կազմակերպումը: Նորմալացման ընթացքում տվյալները բաժանվում են տարբեր աղյուսակների այնպես, որպեսզի յուրաքանչյուր աղյուսակ պահպանի միայն անհրաժեշտ տվյալները:

Նորմալացման հիմնական մակարդակներն են

Առաջին նորմալ ձև (1NF) – աղյուսակում չպետք է լինեն կրկնվող դաշտեր կամ բազմարժեք տվյալներ:

Երկրորդ նորմալ ձև (2NF) – յուրաքանչյուր ոչ բանալի դաշտ պետք է ամբողջությամբ կախված լինի առաջնային բանալուց:

Երրորդ նորմալ ձև (3NF) – աղյուսակում չպետք է լինեն ոչ ուղիղ կախվածություններ ոչ բանալի դաշտերի միջև:

Նորմալացման կիրառումը թույլ է տալիս ստեղծել արդյունավետ և ճիշտ կառուցվածքային տվյալների հենք: PTMS համակարգում բոլոր աղյուսակները բերված են 3NF մակարդակին: Բացառություն կազմող միակ դեպքը activities աղյուսակն է, որտեղ JSONB սյունակում պահվում են փոփոխությունների old_values և new_values արժեքները՝ աուդիտի նպատակով:

Օգտագործված գործիքներ և տեխնոլոգիաներ

PTMS համակարգի մշակման ընթացքում կիրառվել են մի շարք ծրագրային գործիքներ և տեխնոլոգիաներ, որոնք ապահովել են համակարգի ստեղծման, կառավարման, տվյալների բազայի սպասարկման և տարբերակների վերահսկման գործընթացները:

PostgreSQL 16 — տվյալների բազայի սերվեր

Հիմնական պահեստավորման համակարգը. ապահովում է տրանզակցիոնություն (ACID), MVCC բազմատարբերակային միաժամանակյա հսկողություն, JSONB-ի աջակցություն, լրացուցիչ ինդեքսների տեսակներ (B-tree, GIN, GiST, BRIN) և PL/pgSQL ընթացակարգային լեզու:

FastAPI և SQLAlchemy 2 (async) — backend

Backend-ը մշակվել է Python 3.11-ով՝ FastAPI ֆրեյմվորքի վրա հիմնված: Տվյալների բազայի հետ աշխատանքի համար օգտագործվում է SQLAlchemy ORM-ի 2.x տարբերակը՝ ասինխրոն ռեժիմով (asyncpg drivers): Pydantic v2-ը կիրառվում է մուտքային և ելքային տվյալների վավերացման համար, իսկ Alembic-ը՝ տվյալների բազայի սխեմայի միգրացիաների կառավարման համար:

React 18 + TypeScript + Vite — frontend

Frontend-ը մշակվել է React 18-ով՝ TypeScript-ի և Vite build-գործիքի հետ: Տվյալների կառավարման համար օգտագործվում է TanStack Query (հարցումների cache, optimistic updates), հավելվածի վիճակի համար՝ Zustand, ռճավորման համար՝ Tailwind CSS, գրաֆիկների համար՝ Recharts և React Flow:

Git և GitHub — տարբերակների կառավարում

Git-ի միջոցով կոդի թարմացման համար օգտագործվում են git pull, git fetch, git rebase գործողությունները, որոնք թույլ են տալիս հեռակա repository-ից ներբեռնել վերջին փոփոխությունները և համադրել դրանք տեղական նախագծի հետ: Գործընթացի ընթացքում համակարգը կապվում է GitHub-ի origin/main ճյուղին և ստանում վերջին commit-ները, ինչի արդյունքում իրականացվում է ավտոմատ համադրում (fast-forward merge): Այս գործընթացը կարևոր է թիմային աշխատանքի համար, քանի որ ապահովում է կոդի համաժամեցում և վերջին թարմացումների հասանելիություն բոլոր անդամների համար:

Docker և Docker Compose — միջավայրի կազմակերպում

Ամբողջ համակարգը փաթեթավորված է Docker-ի հետ: docker-compose.yml ֆայլում նկարագրված են հինգ ծառայություններ՝ PostgreSQL 16, Redis 7, FastAPI backend, Vite frontend և Nginx reverse-proxy: Մեկ docker compose up --build հրամանով կարելի է բարձրացնել ամբողջ հավելվածը տեղական մեքենայի վրա առանց ձեռքով կախվածությունների տեղադրման:

Alembic migration և seed հրամանների իրականացում

Alembic CLI միջավայրում migration հրամանների կատարման համար backend

պանակում օգտագործվում են `alembic upgrade head` և `alembic revision --autogenerate` հրամանները տվյալների բազայի կառուցվածքը կառավարելու համար: Սկզբում կատարվում է `alembic upgrade head` հրամանը, որի միջոցով Alembic-ը ստուգում և կիրառում է միգրացիոն ֆայլերը: Տերմինալում երևում է, որ տվյալների բազան արդեն թարմացված է (`Database schema is up to date`): Այնուհետև անհրաժեշտության դեպքում օգտագործվում է `alembic downgrade -1` հրամանը, որը նախատեսված է վերջին միգրացիան հետ վերադարձնելու համար: Այս գործընթացը կարևոր է տվյալների բազայի փոփոխությունները վերահսկելու և անհրաժեշտության դեպքում դրանք չեղարկելու համար:

Seed-ի համար մշակվել է առանձին `scripts/seed.py` սցենարը, որը նախատեսված է տվյալների բազայում նախնական տվյալներ ներմուծելու համար: Տերմինալում երևում է, որ համակարգը հերթականությամբ ստեղծում է `superadmin` օգտատեր, `demo` նախագիծ, թեգեր և մի քանի օրինակային առաջադրանքներ՝ կապված `users`, `projects`, `tasks`, `tags` և `comments` աղյուսակների հետ: Այս գործընթացները կարևոր դեր ունեն `backend` համակարգերի մշակման ընթացքում, քանի որ միգրացիաները վերահսկում են տվյալների բազայի կառուցվածքային փոփոխությունները, իսկ `seed`-երը ապահովում են նախնական կամ փորձնական տվյալների ավտոմատ ներմուծումը:

PostgreSQL սերվերի գործարկում Docker-ով

PostgreSQL սերվերի գործարկման գործընթացը `docker-compose` միջավայրում իրականացվում է `docker compose up -d postgres` հրամանի միջոցով, որի միջոցով գործարկվում է PostgreSQL տվյալների բազայի սերվերը: Տերմինալում երևում են PostgreSQL-ի LOG հաղորդագրությունները, որտեղ նշվում է, որ համակարգը վերականգնում է նախորդ աշխատանքային վիճակը (`automatic recovery in progress`) և հաջողությամբ միացնում է սերվերը (`database system is ready to accept connections`):

PostgreSQL psql միջավայրի միացում

PostgreSQL տվյալների բազայի `psql` միջավայրին միացման գործընթացը մուտքագրվում են սերվերի հասցեն (`localhost`), տվյալների բազայի անունը (`ptms`), պորտը (`5432`) և օգտանունը (`ptms_user`), որից հետո համակարգը հաստատում է հաջող միացումը PostgreSQL սերվերին: Տերմինալում երևում են PostgreSQL տարբերակի տվյալները և ակտիվ կապը `localhost` սերվերի հետ: Միացումից հետո կարող ենք կատարել SQL հրամաններ, ստեղծել աղյուսակներ, կատարել հարցումներ և կառավարել տվյալների բազան: Այս գործընթացը կարևոր է տվյալների բազայի կառավարման համար, քանի որ ապահովում է անմիջական կապ PostgreSQL սերվերի հետ և հնարավորություն է տալիս իրականացնել տվյալների մշակում և ադմինիստրացիա:

PostgreSQL տվյալների բազայի աղյուսակների դիտարկում

PostgreSQL տվյալների բազայի աղյուսակների ցուցադրման գործընթացը `psql` միջավայրում իրականացվում է `\dt` հրամանով: Նախ միանում ենք `ptms` տվյալների բազային, այնուհետև `\dt` հրամանը ցուցադրում է բոլոր աղյուսակները, որոնք գտնվում են `public` schema-ում: Արդյունքում համակարգը ցուցադրում է `users`, `projects`, `project_members`, `tasks`, `task_tags`, `tags`, `comments`, `activities` և `alembic_version`

աղյուսակները՝ իրենց owner-ների հետ միասին: Այս գործընթացը կարևոր է տվյալների բազայի կառուցվածքը վերահսկելու համար, քանի որ կարող ենք ստուգել ստեղծված աղյուսակները, համոզվել միգրացիաների հաջող կատարման մեջ և վերահսկել տվյալների կազմակերպվածությունը:

Կոդի խմբագրիչ (Visual Studio Code)

Կուրսային աշխատանքի իրականացման ընթացքում որպես հիմնական կոդային խմբագրիչ օգտագործվել է Visual Studio Code ծրագիրը: Այն ապահովում է արագ, թեթև և հարմարավետ միջավայր ծրագրային կոդի մշակման համար: Ծրագիրը ունի պարզ և հասկանալի ինտերֆեյս, ինչպես նաև սինտաքսիսի գունավորման հնարավորություն, որը հեշտացնում է կոդի ընթերցելիությունը և սխալների հայտնաբերումը: Բացի այդ, VS Code-ը թույլ է տալիս արագ նավիգացիա կատարել ֆայլերի միջև, ունի որոնման հզոր գործիքներ և աջակցում է ընդարձակումների, որոնք ավելացնում են լրացուցիչ ֆունկցիոնալություն Python, Pylance, Prettier, ESLint, GitLens, Docker, Tailwind CSS IntelliSense և այլն: Այս գործիքի կիրառումը նպաստել է ծրագրային կոդի արագ և արդյունավետ մշակմանը, ինչպես նաև աշխատանքի կազմակերպման հեշտացմանը:

Render.com — ամպային տեղակայում

Render-ը ամպային hosting ծառայություն է, որը նախատեսված է վեբ հավելվածների և backend համակարգերի տեղակայման համար: Այն հնարավորություն է տալիս GitHub repository-ի հետ ինտեգրման միջոցով իրականացնել ավտոմատ deployment (CI/CD) գործընթաց: Այն ավտոմատ կերպով տեղադրում է նախագծի կախվածությունները, գործարկում backend ծառայությունները և build է իրականացնում frontend հատվածի համար: Բացի այդ, համակարգը աջակցում է environment variables-ի կառավարմանը առանց .env ֆայլի օգտագործման: Տեղակայման արդյունքում հավելվածը ստանում է առանձին URL հասցե՝ <https://ptms-xzau.onrender.com/>, որի միջոցով համակարգը հասանելի է դառնում ինտերնետում:

Ծրագրային կոդեր

Backend

Սա սերվերի կոդն է, որը պատասխանատու է տվյալների մշակման և բազայի հետ աշխատելու համար: Երբ Frontend-ը հարցում է ուղարկում, այն գնում է routes controller service model database ճանապարհով, հետ քերում տվյալը և վերադարձնում Frontend-ին: Backend-ը ուղղակիորեն չի երևում օգտատիրոջը, բայց ամբողջ տրամաբանությունը հենց այստեղ է՝ authentication, authorization, validation, տվյալների պահպանում: .env ֆայլում պահվում են գաղտնի բանալիները (DB URL, JWT secret), իսկ SQLAlchemy-ն հոգում է SQL հարցումների ստեղծման մասին՝ առանց ձեռքով SQL գրելու:

backend/app/models/project.py

```
class ProjectStatus(str, enum.Enum):
    planning = "planning"
    active = "active"
```

```

on_hold = "on_hold"
completed = "completed"
archived = "archived"

```

```

class ProjectPriority(str, enum.Enum):
    low = "low"
    medium = "medium"
    high = "high"
    critical = "critical"

class Project(Base, UUIDPrimaryKeyMixin, TimestampMixin):
    __tablename__ = "projects"

    title: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    status: Mapped[ProjectStatus] = mapped_column(
        Enum(ProjectStatus, name="project_status"),
        default=ProjectStatus.planning, nullable=False,
    )
    priority: Mapped[ProjectPriority] = mapped_column(
        Enum(ProjectPriority, name="project_priority"),
        default=ProjectPriority.medium, nullable=False,
    )
    owner_id: Mapped[UUID] = mapped_column(
        PG_UUID(as_uuid=True),
        ForeignKey("users.id", ondelete="CASCADE"),
        nullable=False,
    )
    start_date: Mapped[Optional[date]] = mapped_column(Date, nullable=True)
    due_date: Mapped[Optional[date]] = mapped_column(Date, nullable=True)

    owner: Mapped["User"] = relationship(
        back_populates="owned_projects", foreign_keys=[owner_id]
    )
    members: Mapped[List["ProjectMember"]] = relationship(
        back_populates="project", cascade="all, delete-orphan"
    )
    tasks: Mapped[List["Task"]] = relationship(
        back_populates="project", cascade="all, delete-orphan"
    )

```

backend/app/models/task.py

```

class TaskStatus(str, enum.Enum):
    backlog = "backlog"
    todo = "todo"
    in_progress = "in_progress"
    in_review = "in_review"
    done = "done"
    cancelled = "cancelled"

```

```

class Task(Base, UUIDPrimaryKeyMixin, TimestampMixin):
    __tablename__ = "tasks"
    __table_args__ = (
        Index("idx_tasks_project_id", "project_id"),
        Index("idx_tasks_assignee_id", "assignee_id"),
        Index("idx_tasks_status", "status"),
        Index("idx_tasks_due_date", "due_date"),
    )

    title: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    status: Mapped[TaskStatus] = mapped_column(
        Enum(TaskStatus, name="task_status"),
        default=TaskStatus.backlog, nullable=False,
    )
    project_id: Mapped[UUID] = mapped_column(
        PG_UUID(as_uuid=True),
        ForeignKey("projects.id", ondelete="CASCADE"),
    )
    assignee_id: Mapped[Optional[UUID]] = mapped_column(
        PG_UUID(as_uuid=True),
        ForeignKey("users.id", ondelete="SET NULL"),
        nullable=True,
    )
    reporter_id: Mapped[UUID] = mapped_column(
        PG_UUID(as_uuid=True),
        ForeignKey("users.id", ondelete="CASCADE"),
    )
    estimated_hours: Mapped[Optional[Decimal]] = mapped_column(
        Numeric(8, 2), nullable=True,
    )
    due_date: Mapped[Optional[date]] = mapped_column(Date, nullable=True)
    position: Mapped[int] = mapped_column(Integer, default=0, nullable=False)

```

backend/app/api/routes/projects.py

```

@router.get("", response_model=List[ProjectRead])
async def list_projects(
    status_filter: Optional[ProjectStatus] = Query(default=None, alias="status"),
    priority: Optional[ProjectPriority] = None,
    search: Optional[str] = None,
    session: AsyncSession = Depends(get_session),
    current_user: User = Depends(get_current_user),
) -> List[Project]:
    query = (
        select(Project)
        .options(selectinload(Project.owner))
        .order_by(Project.created_at.desc())
    )

    if current_user.role not in (UserRole.superadmin, UserRole.admin):
        member_subq = (
            select(ProjectMember.project_id)
            .where(ProjectMember.user_id == current_user.id)

```

```

        .scalar_subquery()
    )
    query = query.where(or_(
        Project.owner_id == current_user.id,
        Project.id.in_(member_subq),
    ))
    if status_filter:
        query = query.where(Project.status == status_filter)
    if priority:
        query = query.where(Project.priority == priority)
    if search:
        like = f"%{search}%"
        query = query.where(or_(
            Project.title.ilike(like),
            Project.description.ilike(like),
        ))

    result = await session.execute(query)
    return list(result.scalars().all())

```

backend/app/api/routes/tasks.py

```

@router.post(
    "/projects/{project_id}/tasks",
    response_model=TaskRead,
    status_code=status.HTTP_201_CREATED,
)
async def create_task(
    project_id: UUID,
    payload: TaskCreate,
    session: AsyncSession = Depends(get_session),
    current_user: User = Depends(get_current_user),
) -> Task:
    project = await _user_can_access_project(
        session, project_id, current_user,
    )
    if not project:
        raise HTTPException(status_code=404, detail="Project not found")

    task = Task(
        title=payload.title,
        description=payload.description,
        status=payload.status,
        priority=payload.priority,
        project_id=project_id,
        reporter_id=current_user.id,
        assignee_id=payload.assignee_id,
        due_date=payload.due_date,
    )
    session.add(task)
    await session.flush()
    await record_activity(
        session,
        entity_type="task",

```

```

        entity_id=task.id,
        action="created",
        user_id=current_user.id,
        new_values={"title": task.title},
    )
    await session.commit()
    return task

```

backend/alembic/versions/0001_init.sql (trigger)

```

-- Trigger: updated_at column auto-refresh
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER projects_set_updated_at
BEFORE UPDATE ON projects
FOR EACH ROW
EXECUTE PROCEDURE set_updated_at();

CREATE TRIGGER tasks_set_updated_at
BEFORE UPDATE ON tasks
FOR EACH ROW
EXECUTE PROCEDURE set_updated_at();

-- Activity log trigger
CREATE OR REPLACE FUNCTION log_task_status_change()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status IS DISTINCT FROM OLD.status THEN
        INSERT INTO activities (entity_type, entity_id, action,
                                user_id, old_values, new_values)
        VALUES ('task', NEW.id, 'status_changed', NEW.reporter_id,
                jsonb_build_object('status', OLD.status),
                jsonb_build_object('status', NEW.status));
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tasks_log_status_change
AFTER UPDATE ON tasks
FOR EACH ROW
EXECUTE PROCEDURE log_task_status_change();

```

backend/app/api/routes/auth.py

```

@router.post("/login", response_model=TokenResponse)
async def login(
    payload: LoginRequest,

```

```

        response: Response,
        session: AsyncSession = Depends(get_session),
    ) -> TokenResponse:
        result = await session.execute(
            select(User).where(User.email == payload.email)
        )
        user = result.scalar_one_or_none()
        if not user or not verify_password(
            payload.password, user.hashed_password,
        ):
            raise HTTPException(
                status_code=401, detail="Invalid credentials",
            )
        if not user.is_active:
            raise HTTPException(status_code=403, detail="Inactive user")

        access = create_access_token(
            str(user.id), {"role": user.role.value},
        )
        refresh = create_refresh_token(str(user.id))
        _set_refresh_cookie(response, refresh)
        return TokenResponse(
            access_token=access,
            user=UserPublic.model_validate(user),
        )

```

backend/app/main.py

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.api.routes import (
    auth, comments, dashboard,
    db_inspector, projects, tasks, users,
)
from app.middleware.query_interceptor import QueryInterceptorMiddleware

app = FastAPI(title="PTMS API", version="1.0.0")
app.add_middleware(CORSMiddleware,
    allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])
app.add_middleware(QueryInterceptorMiddleware)

app.include_router(auth.router, prefix="/api")
app.include_router(users.router, prefix="/api")
app.include_router(projects.router, prefix="/api")
app.include_router(tasks.router, prefix="/api")
app.include_router(comments.router, prefix="/api")
app.include_router(dashboard.router, prefix="/api")
app.include_router(db_inspector.router, prefix="/api")

```

backend/requirements.txt

```

fastapi==0.115.5
uvicorn[standard]==0.32.1

```



```

sqlalchemy[asyncio]==2.0.36
asynccpg==0.30.0
alembic==1.14.0
pydantic[email]==2.10.3
pydantic-settings==2.6.1
passlib[bcrypt]==1.7.4
pyjwt==2.10.1
redis==5.2.0
python-multipart==0.0.19

```

Frontend

Սա օգտատիրոջ տեսած մասն է: React-ը կառուցում է UI-ը փոքր components-երից, react-router-dom-ը կառավարում է, թե որ էջն է բացվում, axios-ը Backend-ին հարցումներ է ուղարկում, TanStack Query-ն cache-ում է պատասխանները, իսկ Vite-ը վերջում բոլոր ֆայլերը հավաքում է մեկ bundle-ի մեջ:

frontend/src/pages/ProjectsPage.tsx

```

export function ProjectsPage() {
  const qc = useQueryClient();
  const [creating, setCreating] = useState(false);
  const [title, setTitle] = useState("");
  const [description, setDescription] = useState("");

  const { data: projects = [], isLoading } = useQuery({
    queryKey: ["projects"],
    queryFn: () => projectsApi.list(),
  });

  const createMutation = useMutation({
    mutationFn: () => projectsApi.create({ title, description }),
    onSuccess: () => {
      qc.invalidateQueries({ queryKey: ["projects"] });
      setCreating(false);
      setTitle("");
      setDescription("");
    },
  });

  return (
    <div className="space-y-6">
      <div className="flex items-center justify-between">
        <h1 className="text-2xl font-semibold">Projects</h1>
        <button onClick={() => setCreating(true)}
          className="btn-primary">
          <Plus className="h-4 w-4" />
          New Project
        </button>
      </div>
      { /* ... project list grid ... */ }
    </div>
  );
}

```

frontend/src/api/endpoints.ts

```
export const projectsApi = {
  list: (params?: { status?: string; priority?: string; search?: string }) =>
    api.get<Project[]>("/projects", { params }).then((r) => r.data),
  get: (id: string) =>
    api.get<ProjectDetail>(`/projects/${id}`).then((r) => r.data),
  create: (data: Partial<Project>) =>
    api.post<Project>("/projects", data).then((r) => r.data),
  update: (id: string, data: Partial<Project>) =>
    api.patch<Project>(`/projects/${id}`, data).then((r) => r.data),
  remove: (id: string) => api.delete(`/projects/${id}`),
};

export const tasksApi = {
  listByProject: (projectId: string, params?: Record<string, string>) =>
    api.get<Task[]>(`/projects/${projectId}/tasks`, { params })
      .then((r) => r.data),
  get: (id: string) =>
    api.get<TaskDetail>(`/tasks/${id}`).then((r) => r.data),
  create: (projectId: string, data: Partial<Task>) =>
    api.post<Task>(`/projects/${projectId}/tasks`, data)
      .then((r) => r.data),
  update: (id: string, data: Partial<Task>) =>
    api.patch<Task>(`/tasks/${id}`, data).then((r) => r.data),
  move: (id: string, status: string, position: number) =>
    api.patch<Task>(`/tasks/${id}/move`, { status, position })
      .then((r) => r.data),
};
```

frontend/src/api/client.ts

```
import axios from "axios";
import { useAuthStore } from "@store/auth-store";

export const api = axios.create({
  baseURL: "/api",
  withCredentials: true,
});

api.interceptors.request.use((config) => {
  const token = useAuthStore.getState().accessToken;
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

api.interceptors.response.use(
  (r) => r,
  async (error) => {
    if (error.response?.status === 401) {
      useAuthStore.getState().logout();
    }
  }
);
```

```

    return Promise.reject(error);
  },
);

```

frontend/src/App.tsx

```

function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <BrowserRouter>
        <Routes>
          <Route path="/login" element={<LoginPage />} />
          <Route path="/register" element={<RegisterPage />} />
          <Route
            path="/"
            element={
              <ProtectedRoute>
                <AppLayout />
              </ProtectedRoute>
            }
          />
          <Route index element={<DashboardPage />} />
          <Route path="projects" element={<ProjectsPage />} />
          <Route path="projects/:id" element={<ProjectDetailPage />} />
          <Route path="tasks/:id" element={<TaskDetailPage />} />
          <Route path="db-explorer" element={<DbExplorerPage />} />
        </Routes>
      </BrowserRouter>
    </QueryClientProvider>
  );
}

```

frontend/package.json

```

{
  "name": "ptms-frontend",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build",
    "preview": "vite preview",
    "lint": "eslint src --ext .ts,.tsx"
  },
  "dependencies": {
    "@tanstack/react-query": "^5.62.0",
    "axios": "^1.7.9",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router-dom": "^6.28.0",
    "reactflow": "^11.11.4",
    "recharts": "^2.13.3",
    "zustand": "^5.0.2"
  }
}

```

```
}
}
```

Եզրակացություն

Սույն կուրսային աշխատանքի շրջանակներում նախագծվել և մշակվել է նախագծերի և առաջադրանքների կառավարման համակարգի (Project & Task Management System, PTMS) տվյալների հենքը: Աշխատանքի ընթացքում ուսումնասիրվել են տվյալների հենքերի նախագծման հիմնական սկզբունքները, հարաբերական տվյալների հենքի կառուցվածքային տարրերը, ինչպես նաև PostgreSQL և MySQL կառավարման համակարգերի հիմնական հնարավորությունները:

Նախագծված համակարգը ապահովում է թիմային աշխատանքի արդյունավետ կազմակերպումը՝ թույլ տալով պահել և կառավարել նախագծերը, դրանց անդամներին, առաջադրանքները, ենթաառաջադրանքները, առաջնահերթությունները, կարգավիճակները, թեգերը, մեկնաբանությունները և գործողությունների աուդիտը: Տվյալների հենքի կառուցվածքը ներառում է «users, projects, project_members, tasks, task_tags, tags, comments, activities» և այլ աղյուսակներ, որոնք փոխկապակցված են առաջնային և արտաքին բանալիների միջոցով:

Տվյալների ամբողջականությունն ու անվտանգությունն ապահովվում են «Trigger»-ների (updated_at-ի ավտոմատ թարմացում, գործողությունների աուդիտի գրառում), «Stored Function»-ների (PL/pgSQL ընթացակարգեր) և «View»-ների (v_project_progress նախագծի առաջընթացի համար) կիրառման միջոցով: Կատարվել է նաև տվյալների նորմալացում՝ 1NF, 2NF, 3NF մակարդակների պահանջներին համապատասխան, ինչը նպաստել է տվյալների կրկնության վերացմանը և համակարգի արդյունավետության բարձրացմանը:

Գործնական մասում կիրառվել են ժամանակակից ծրագրային գործիքներ՝ «Python, FastAPI, SQLAlchemy 2 (async), Alembic, Pydantic v2, PostgreSQL 16, Redis 7, React 18, TypeScript, Vite, TanStack Query, Tailwind CSS, Docker Compose, Git» և «Visual Studio Code», որոնք ապահովել են թիմային աշխատանքի կազմակերպումը, կոդի տարբերակների վերահսկումը և տվյալների բազայի միգրացիաների կառավարումը:

Ամփոփելով՝ կարելի է արձանագրել, որ կուրսային աշխատանքի նպատակը հաջողությամբ իրականացվել է. նախագծված տվյալների հենքը բավարարում է ժամանակակից նախագծերի կառավարման տեղեկատվական համակարգերին ներկայացվող հիմնական պահանջներին՝ ապահովելով տվյալների արագ հասանելիություն, պահպանություն, ամբողջականություն և անվտանգություն:

Գրականության ցանկ

<https://www.postgresql.org/>

<https://en.wikipedia.org/wiki/PostgreSQL>

<https://www.ibm.com/think/topics/postgresql-vs-mysql>

<https://www.geeksforgeeks.org/postgresql/postgre-window-functions/>

<https://www.geeksforgeeks.org/postgresql/working-with-regular-expressions-in-postgresql/>

<https://www.mongodb.com/resources/basics/databases/database-migration>

<https://docs.sqlalchemy.org/en/20/>
<https://fastapi.tiangolo.com/>
<https://alembic.sqlalchemy.org/en/latest/>
<https://react.dev/>
<https://tanstack.com/query/latest>
<https://jwt.io/introduction>